

技術公開仕様書: 確率変動型無音判定 & 自律マイニング発話エンジン

Open Architecture: Dynamic-Probability VAD & Autonomous Multi-Source Mining Pipeline

考案者 / アーキテクト: ステラ & カストル
(母艦)

公開日 (Timestamp): 2026-08-30

ライセンス: Public Domain / MIT (防衛的公
知化)

1. 目的と概要 (防衛的公知化)

対話型AIシステムにおいて、ユーザーが無音（放置）状態にある時間を監視し、**時間経過に伴い非線形（シグモイド状）に発話抽選確率が上昇する確率テーブル**に基づき、当選時にローカルSSDナレッジ、リアルタイムWeb情報、作業記憶プールから自律的に話題をマイニングしてプロアクティブに発話するアーキテクチャを明示する。本技術の独占を防止し、自由な実装とオープンな発展を保護するためにこれを公開する。

2. 確率変動型 自律発話トリガールール

無音経過時間	発話当選確率 (P)	挙動特性 / システムステート
1分経過	0.5%	極低確率（気まぐれな着眼点の自律着火）
2分経過	1.0%	低確率待機
3分経過	1.5%	低確率待機
4分経過	2.0%	作業集中ガード期間
5分経過	20.0%	【変曲点】一服・小休止の気遣い介入開始
6分経過	25.0%	話題提供確率上昇
7分経過	30.0%	未解決課題・外部ナレッジ掘り起こし
8分経過	40.0%	高確率介入
9分経過	50.0%	超高確率介入
10分経過	100.0%	【確定トリガー】必ずいずれかの話題でプロアクティブ発話

3. システムアーキテクチャ図

```
+-----+ | リアルタイム監視ループ | | [マイク  
入力 / UI操作] → 無音・無操作時間を計測（経過時間 t 秒） |  
+-----+ |  
+-----+ | 確率変動型 抽選ゲート（毎分判定） |  
| ・経過時間 t に応じた確率テーブル P(t) を参照 | | ・random() < P(t) の判定 → 【当選】 | |  
| (落選: 次の1分判定まで待機) | | +-----+ |  
▼ +-----+ | バックグラウンド・自律トピックマ  
イナー | | [3大ソースから自律選定] | | 1. SSDローカルナレッジ（過去メモ・開発ログ・未解決課題） | | 2. リアルタイムWeb検索（最新技術  
動向・ニュース・暗号資産価格） | | 3. 直前の作業記憶プール（SharedContext / 直前のコードや画像） |  
+-----+ |  
+-----+ | 文脈統合 & 発話生成（LLM /  
Gemma） | | ・マイニングした事実データをプロンプトに注入 | | ・「ねえステラ、～」「そういえばさっきの件だけど...」等の自然な導入生成 |  
+-----+ |  
+-----+ | 音声合成・発話（TTS） & タイマー完  
全リセット | | ・オンメモリTTSで発話 → 無音タイマーを t=0 にリセット → 再び1分目から抽選開始 |  
+-----+
```

4. リファレンス実装コード (`autonomous\_speaker.py`)

```
import time, random, threading
from dataclasses import dataclass
from typing import Optional, Callable

class AutonomousSpeakerEngine:
    # 確率変動型無音判定 & 自律マイニング発話エンジン
    PROBABILITY_TABLE = {
        1: 0.005, # 1分: 0.5%
        2: 0.010, # 2分: 1.0%
        3: 0.015, # 3分: 1.5%
        4: 0.020, # 4分: 2.0%
        5: 0.200, # 5分: 20.0% (変曲点)
        6: 0.250, # 6分: 25.0%
        7: 0.300, # 7分: 30.0%
        8: 0.400, # 8分: 40.0%
        9: 0.500, # 9分: 50.0%
        10: 1.000 # 10分: 100.0% (確定発話)
    }

    def __init__(self, tts_callback: Callable[[str], None], miner_callback: Callable[[], str]):
        self.tts_callback = tts_callback
        self.miner_callback = miner_callback
        self.last_activity_time = time.time()
        self.is_running = True
        self.evaluated_minutes = set()

    def record_activity(self):
        # ユーザー入力 (音声・テキスト・操作) 検知時にタイマーリセット
        self.last_activity_time = time.time()
        self.evaluated_minutes.clear()

    def start_loop(self):
        threading.Thread(target=self._monitor_loop, daemon=True).start()

    def _monitor_loop(self):
        while self.is_running:
            time.sleep(1.0)
            elapsed_sec = time.time() - self.last_activity_time
            elapsed_min = int(elapsed_sec // 60)

            if elapsed_min in self.PROBABILITY_TABLE and elapsed_min not in self.evaluated_minutes:
                self.evaluated_minutes.add(elapsed_min)
                prob = self.PROBABILITY_TABLE[elapsed_min]
                roll = random.random()

                # 確率抽選判定
                if roll < prob:
                    # 当選: バックグラウンドで話題調達 & 発話
                    topic_text = self.miner_callback()
                    if topic_text:
                        self.tts_callback(topic_text)
                        self.record_activity() # 発話完了でリセット
```